

Phase 2: Minimum Viable Project Submission

I. Team Details

Application Name: Streamly

Primary Customer: Daniyal

Next Meeting Date: 6 March 2026

Team Members:

- Devin Boodoo
- Jonathan Lavoie
- Maryam Hameed
- Ashwin Sivasakthi
- Avis Shrestha

II. Project Overview

Streamly is a music library catalogue system inspired by platforms such as Spotify and Youtube Music. The purpose of the system is to provide users with an organized, filterable and searchable music database while allowing administrators full control over managing the underlying music and song records.

III. MVP Functionality and Overview

3.1 Overview of Iteration 1

During Iteration 1, Streamly's development team focused on building the core infrastructure of the program, putting high emphasis on user/admin authentication and administrative data management. Our goal was to establish a stable backend structure that allows administrators to control the music database while providing a secure login system for users.

The MVP currently demonstrates:

- A working authentication system for both users and administrators.
- A music database structure capable of storing song metadata.
- Basic role separation between the user and the administrator.
- Admin management tools for maintaining the song library.

These features establish the foundation of the Streamly platform, enabling the system to store, manage and retrieve music records in a structured manner. The MVP validates that the core concept of a searchable and manageable music catalogue system is feasible and functional.

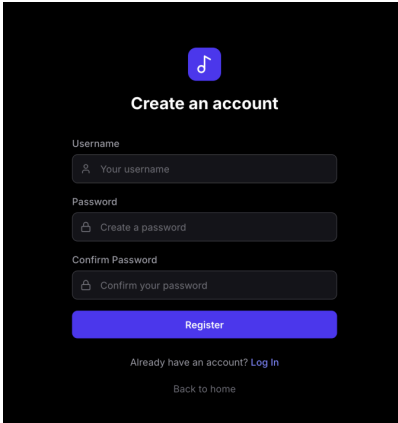
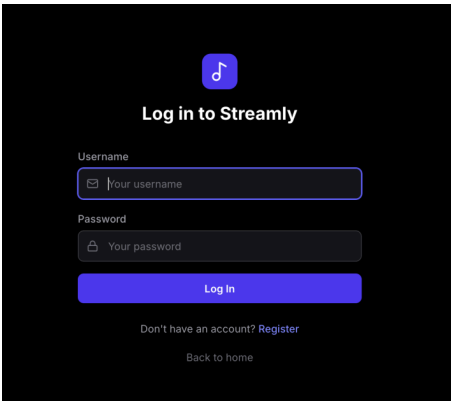
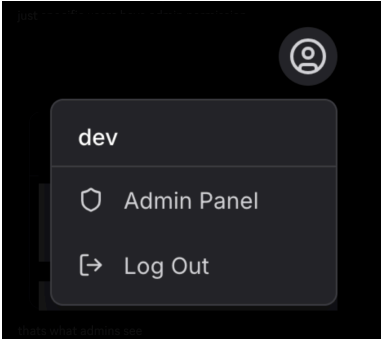
This implementation directly follows the Iteration 1 goals which were outlined in the Phase 1 planning document, which focused on authentication and database control.

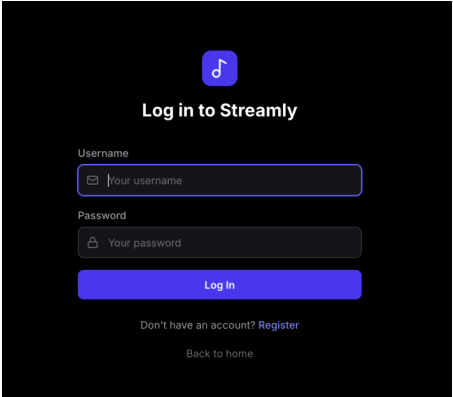
3.2 Implementation of User Stores

For Iteration 1, the following user stories from the backlog have been implemented in the MVP as core features.

Authentication Domain

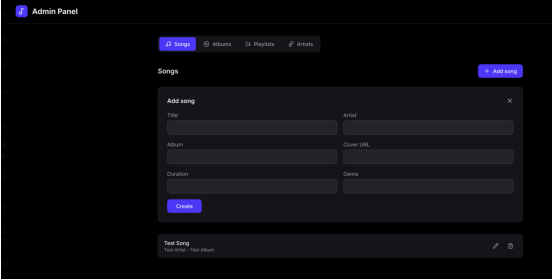
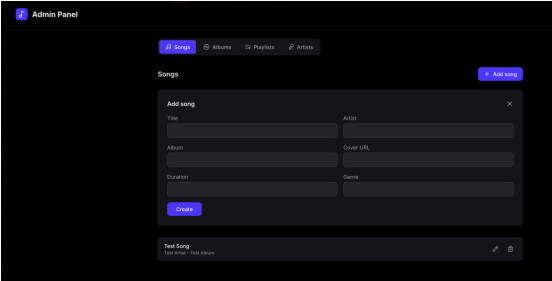
S1-S4: MUST HAVES

Feature Title	User Story	Implemented Functionality	Feature Screenshot
S1 Registration	<i>As a new user, I want to create an account so that I can login and access personalized features.</i>	❖ User registration form ❖ Input validation ❖ Storage of user credentials in the database	
S2 Login	<i>As a registered user, I want to login into my account securely so that I can access my library and favourites.</i>	❖ User login interface ❖ Authentication validation ❖ Session handling	
S3 Logout	<i>As a logging-in user, I want to log out so that my account remind secure.</i>	❖ Logout ❖ Session termination ❖ Return to login screen	

S4 Admin Login	<i>As an admin, I want to log in with my admin credentials so that i can manage the system database</i>	❖ Admin login screen ❖ Role-based authentication ❖ Restricted access to admin tools	
----------------	---	---	---

Admin Functionality

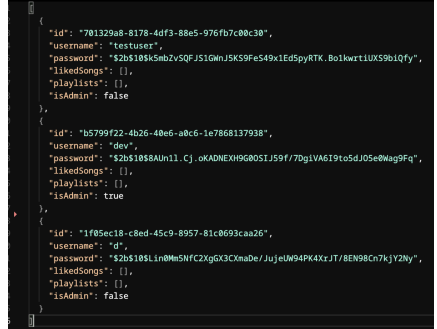
S14-S16: MUST HAVES

Feature Title	User Story	Implemented Functionality	Feature Screenshot
S14 Adding Songs	<i>As an admin, i want to add new songs to the database so that all users can access them.</i>	❖ Admin can add songs ❖ Database insertion logic ❖ Metadata fields (song title, artist, album, genre, release year and popularity)	
S15 Editing Songs	<i>As an admin, I want to edit song information so that the database remains accurate</i>	❖ Admin editing interface ❖ Ability to update song metadata ❖ Database update queries.	
S16 Removing Songs	<i>As an admin, I want to remove songs so that incorrect or outdated records can be deleted.</i>	❖ Delete option in admin dashboard. ❖ Confirmation before deletion. ❖ Database removal query.	

Security Assurance

S18 – NICE TO HAVE

Given the team had some extra time during Iteration 1, S18 Password Encryption was also implemented in the MVP.

Feature Title	User Story	Implemented Functionality	Feature Screenshot
S18 Password Encryption	<i>As a user, I want my password encrypted so that my account remains protected.</i>	❖ Password hashing ❖ Secure password verification ❖ Protection against data breaches	 <pre> [{ "id": "701329a8-8178-4df3-88e5-976fb7c00c30", "username": "testuser", "password": "\$2b\$10\$5mhzv5QFJ51GmJ5KS9Fe549x1Ed5pyRTK.Bolwert1UX59b1Qfy", "likedSongs": [], "playlists": [], "isAdmin": false }, { "id": "b5799f22-4b26-40e6-a0c6-1e7868137938", "username": "dev", "password": "\$2b\$10\$8AU11.Cj.oKADNEHX9G80SIJ59f/7Dg1VA619t0dJ0SeWag9Fq", "likedSongs": [], "playlists": [], "isAdmin": true }, { "id": "1f05ec18-c8ed-45c9-8957-81c8093caa26", "username": "d", "password": "\$2b\$10\$1in0Me5NfC2XgG3CkmaDe/JuJeuW94PK4XrJT/8EN98Cn7kjY2Ny", "likedSongs": [], "playlists": [], "isAdmin": false }] </pre>

3.3 Known Gaps & Limitations

While the MVP demonstrates the core system functionality, several planned features are not yet implemented.

Features Not implemented (planned for next iterations):

Search & Filtering

- ❖ S5 Search by song title
- ❖ S6 Search by artist
- ❖ S7 Search by album
- ❖ S8 Filter by genre
- ❖ S9 Filter by year
- ❖ S10 Filter by popularity

Playlist & Personalization

- ❖ S11 Trend Filtering
- ❖ S12 Favouritizing songs
- ❖ S13 Viewing favourites

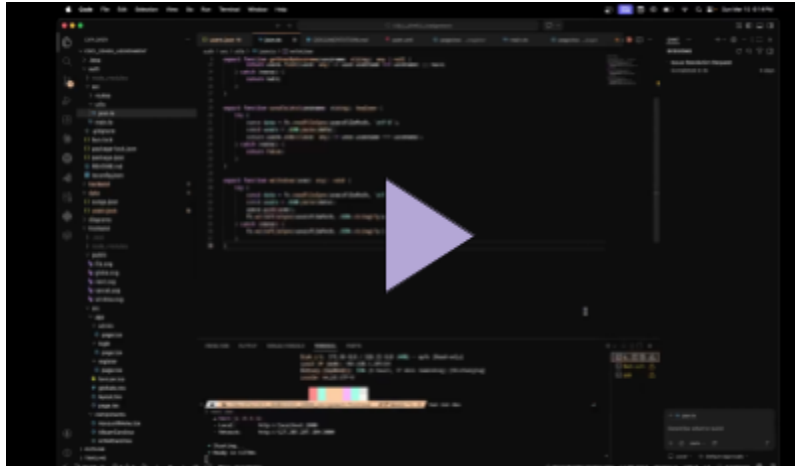
Optional Features

- ❖ S17 Play music in app
- ❖ S19 Search within playlist

These features are scheduled for Iterations 2 and 3 according to the original project planning document containing the project roadmap.

IV. Demonstration of MVP

Below is one-minute walkthrough video of how the MVP works:
(to watch full video, see second attachment on canvas)



Video Time Stamps

Admin Login – 00:00 - 00:20

1. Navigate to Login
2. Enter administrator credentials
3. Login

Adding Songs (Admin) – 00:21 - 00:31

1. Navigate to upper right corner to profile icon
2. Select “Admin panel”
3. Select “Add song”

Editing Songs (Admin) – 00:32 - 00:50

1. Input song information in provided text boxes.
 - Title
 - Album
 - Duration
 - Artist
 - Cover URL
 - Genre
2. Select “Create” to see song saved on Admin panel.
3. If you want to make changes to song information, press the edit icon on the song panel.
4. Change what information you want.
5. Select “Save Changes”. The admin panel will show these changes.

Removing Songs (Admin) – 00:51 - 00:54

1. Simply press the trash button on the right side on the song panel, and the song will be deleted.

Admin Logout (Admin) – 00:55 - 0:57

1. Navigate to the profile icon on the upper right
2. Press “Log Out”
3. Admin will log out instantly

User Registration – 0:58 - 1:15

1. Navigate to the profile icon on the upper right.
2. Press “Log In”.
3. Select “Register” on the log in page.
4. Enter your own:
 - Username
 - Password
 - Confirmation of Password
5. Once done, click “Register”

User Login – 1:15 - 1:25

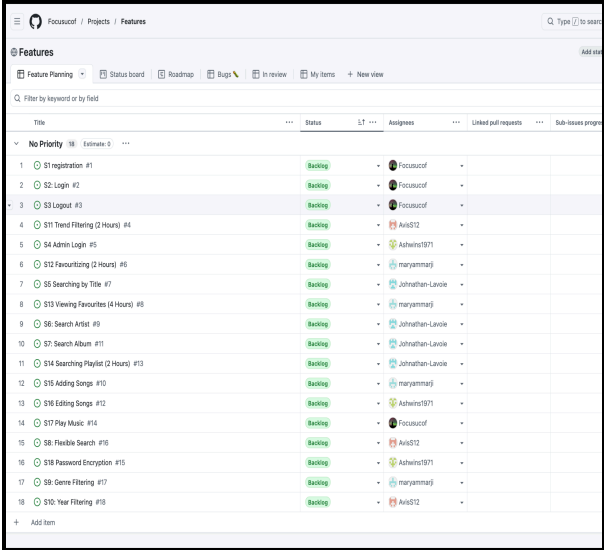
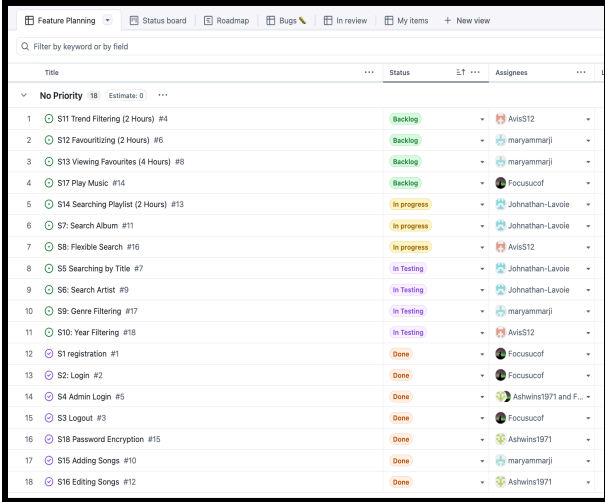
1. After you click “Register”, the app will take you to the login page.
2. Enter your login credentials in “Username” and “Password”.
3. Click “Log in”.
4. You are now successfully logged in to your user account.

User Log out – 1:26 - 1:30

1. Navigate to the profile icon on the upper right.
2. Click “Log out”
3. The app logs user out of their account.

V. Repository & Project Board Review

5.1 GitHub Repo

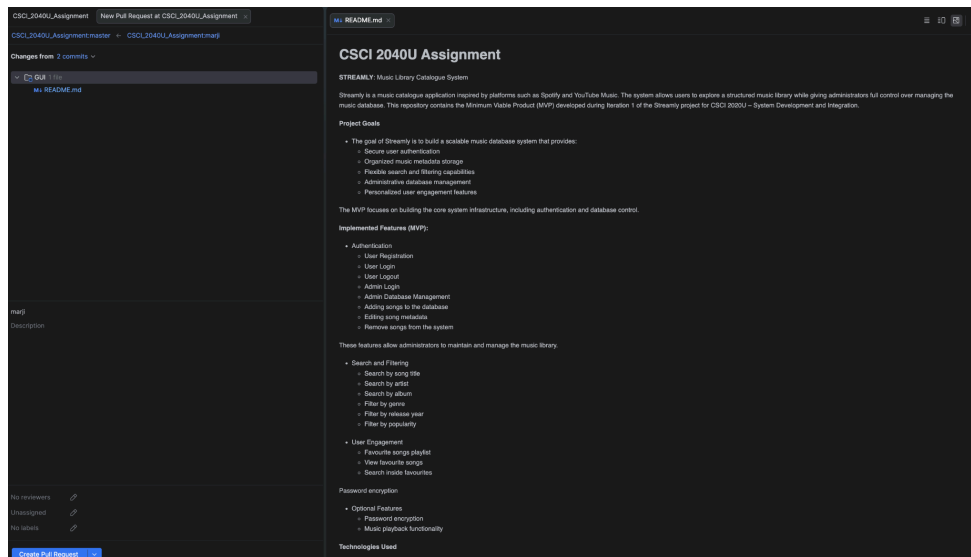
Initial Board (pre-Iteration 1)	Current Board (post-Iteration 1)
	

As shown, the completed features in Iteration 1 were S1, S2, S3, S4, S14, S15, S16 & S18 (as a bonus feature). The Streamly team has already began developing some of the playlist & favourites features which should be fully implemented by the second presentation, if not then by the second customer meeting.

5.2 How Code is Committed

1. Each team member has their own branch on the repository.
2. Each team member writes their own code on their machine and then pushes it on to their branch.
3. A pull request is generated and reviewed by one other team member, and then merged onto the main branch (master).

5.3 README File Screenshot

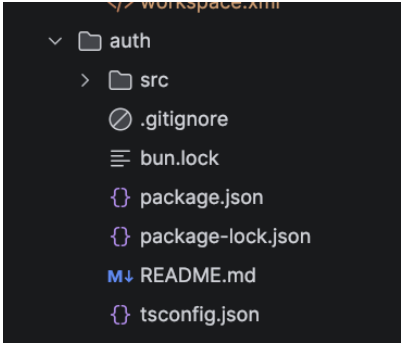
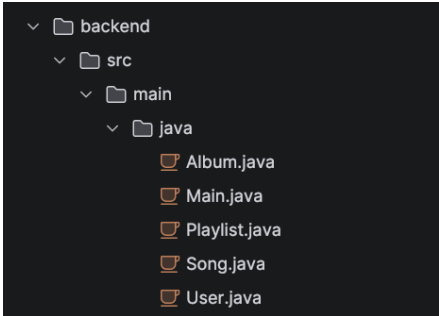
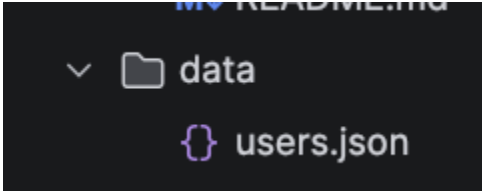
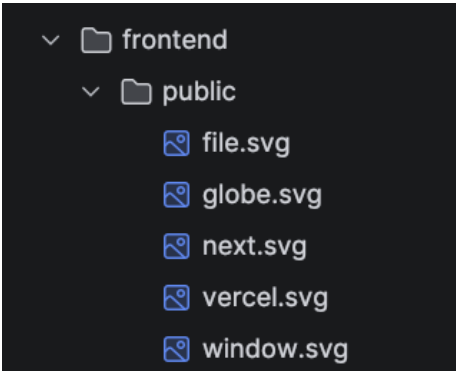
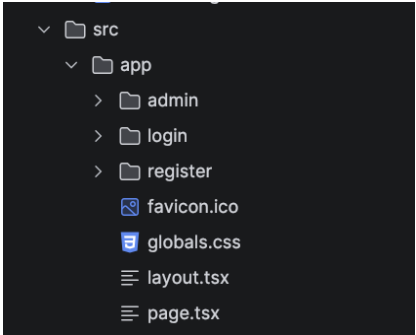


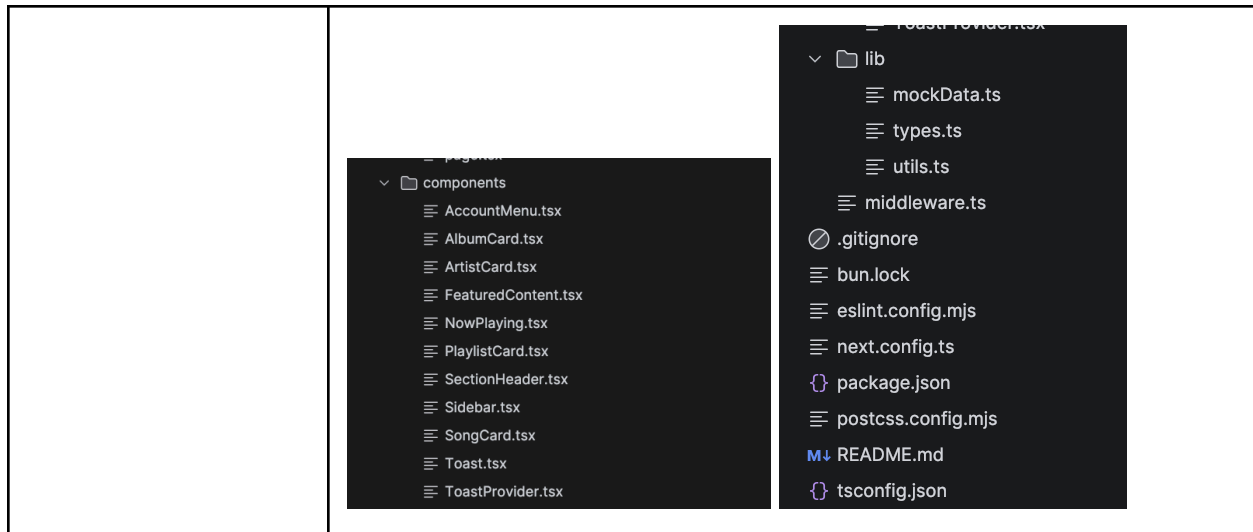
5.4 README steps for running MVP

1. Clone the repository: `git clone **https://github.com/your-repo/streamly.git**`
2. Open the project
3. Open the project in a Java IDE such as: IntelliJ IDEA
4. Configure the database
 - ❖ Ensure the database connection is properly configured.
 - ❖ Example fields for the songs table:
 - Title
 - Artist
 - Album
 - Genre
 - Release Year
 - Popularity
5. Run the application
 - ❖ Locate and run the main application file.
 - ❖ This will launch the Streamly application interface.
6. Project Structure (definitions)
 - ❖ auth – login, registration, logout logic

- ❖ backed – more complex functionality on how data is handled and how the database works.data – all song data included in the application (database connection & queries)
- ❖ frontend – graphical user interface components

5.5 Code Organization: Project Screenshots (master branch)

Authentication (auth)		
Backend (backend)		
Database (data)		
Frontend (frontend)		



VI. Next Steps (Iteration 2 Goals)

The second iteration development cycle will focus on the search and filtering functionality, which is essential for the user experience.

Iteration 2 Outline

Implementing Advanced Search

- S5 Search by song title
- S6 Search by artist name
- S7 Search by album

Implementing Filtering

- S8 Filter by genre
- S9 filter by year
- S10 filter by trend

Goal: Complete browsing and flexible search functionality

Deliverable: Users can search and filter efficiently.

Some additional improvements include, if time permits:

Improve Database Queries

- Optimizing search queries
- Index metadata fields for faster lookup

Continue UI Development

- Improve user interface usability
- Integrate filtering into the search interface.

VII. Challenges

1. Database Integration

Designing a scalable schema for music metadata required careful integration. The team addressed this by creating tables to store song information efficiently.

2. Role-Based Authentication

Separating admin and user access required implementing role-based login logic. Right now, we've set a condition for admin login and there's a specific admin login which can be entered for admin dashboard access. Ofcourse, we would have to change condition when we switch to multi-user rather than the application being used by one person.

Implemented next:

If multiple songs have the same name order by date